

"""

VenturiOperator – the 360-degree evolving coupling differential.

PHYSICS:

A Venturi constricts flow, drops local pressure at the throat,
and draws in injected fluid. In field theory:

pressure → local field energy density $h(x) = |\nabla\psi|^2 + V(|\psi|^2)$

velocity → $|\partial_t \psi|$

throat → annular ring where $|\nabla\psi|^2$ is maximal

injection → domain signal pulled in at the throat

Continuity on the 2D torus (radial): $v \cdot 2\pi r = \text{const}$

→ $v(r) \propto r_0/r$ (beta=1)

Bernoulli: $P + \frac{1}{2}\rho v^2 = \text{const}$

→ gain $G = 1 - (r_{\text{in}}/r_0)^2$

360-DEGREE:

The Venturi is radially symmetric on the torus – no preferred injection
angle. Throat is an annulus at radius r_0 from the field's center of mass.
As the field rotates or drifts, the throat follows.

EVOLUTION:

The throat radius r_0 evolves as a differential driven by the energy
deviation from the Silver Ratio target:

$$dr_0/dt = -\lambda_v \cdot (E - E^*)/E^* \cdot r_0$$

When $E > E^*$: throat contracts → suction increases → more domain pulled in

When $E < E^*$: throat expands → suction decreases → system near closure

CROSS-DOMAIN:

The Venturi couples two domains simultaneously:

GENERIC domain: r_0 is driven by $E = |z|^2/2$ (DC mode energy)

MSR domain: injection amplitude is $\hat{\psi}_{\text{MSR}}$ evaluated at throat

Both flow through the same $V(r, r_0(t))$.

"""

```
from __future__ import annotations
import numpy as np
import math
from dataclasses import dataclass, field
from typing import Optional, Tuple
```

```

from uma.semantic.constants import silver_E_target, INV_DELTA_S_SQ

# -----
# Config
# -----

@dataclass
class VenturiConfig:
    # Throat geometry
    r0_init: float = 0.15      # initial throat radius (torus units 0-0.5)
    r_in: float = 0.04         # inner cutoff (avoids 1/r singularity)
    r_out: float = 0.42        # outer wall
    beta: float = 1.0          # radial velocity exponent (1 = 2D cylindrical)

    # Throat evolution
    lambda_v: float = 0.1      # evolution rate
    r0_min: float = 0.03       # minimum throat radius
    r0_max: float = 0.45       # maximum throat radius

    # Coupling strength
    coupling: float = 1.0      # overall amplitude of cross-domain coupling

    # Physics
    kT: float = 0.005

    @classmethod
    def from_uma_config(cls, cfg, **kwargs) -> "VenturiConfig":
        return cls(kT=cfg.generic.kT, **kwargs)

# -----
# State
# -----

@dataclass
class VenturiState:
    r0: float      # current throat radius
    G: float       # current Bernoulli gain
    E_field: float # current DC mode energy
    t: float       # time
    dr0_dt: float  # current throat velocity
    history: list = field(default_factory=list)

    def record(self):
        self.history.append({
            "t": self.t, "r0": self.r0, "G": self.G,

```

```

        "E": self.E_field, "dr0_dt": self.dr0_dt
    })

# -----
# VenturiOperator
# -----

class VenturiOperator:
    """
    The 360-degree evolving Venturi coupling differential.

    Usage::

        cfg = VenturiConfig.from_uma_config(uma_cfg)
        venturi = VenturiOperator(N=32, cfg=cfg)

        # Each timestep:
        psi_injected = venturi.step(psi_field, psi_inject, z_dc, dt)
        # psi_injected = psi_field + Venturi coupling term
    """

    def __init__(self, N: int, cfg: Optional[VenturiConfig] = None):
        self.N = N
        self.cfg = cfg or VenturiConfig()
        self._state = VenturiState(
            r0=cfg.r0_init if cfg else 0.15,
            G=0.0, E_field=0.0, t=0.0, dr0_dt=0.0,
        )
        self._R = self._build_radial_grid(N)
        self._state.G = self._bernoulli_gain(self._state.r0)

# -----
# Grid
# -----

    @staticmethod
    def _build_radial_grid(N: int) -> np.ndarray:
        """
        Radial distance from the center of the torus (0.5, 0.5),
        with periodic wrapping – so the Venturi is truly 360-degree.
        """
        x = np.linspace(0, 1, N, endpoint=False)
        X, Y = np.meshgrid(x, x)
        dx = np.abs(X - 0.5); dx = np.minimum(dx, 1 - dx)
        dy = np.abs(Y - 0.5); dy = np.minimum(dy, 1 - dy)
        return np.sqrt(dx**2 + dy**2)

```

```

# -----
# Venturi field V(r, r0)
# -----

def _venturi_field(self, r0: float) -> np.ndarray:
    """
     $V(r, r_0) = (r_0 / \max(r, r_{in}))^{\beta} \cdot \text{envelope}(r)$ 

    Peak = 1.0 at  $r = r_0$  (throat).
    Falls to 0 at  $r = r_{out}$  (wall).
    Bounded at  $r < r_{in}$  (inner cutoff).
    """
    c = self.cfg
    r_eff = np.maximum(self._R, c.r_in)
    throat = (r0 / r_eff) ** c.beta
    # Smooth envelope: 1 at throat, 0 at wall
    envelope = np.clip((c.r_out - self._R) / (c.r_out - r0 + 1e-8), 0.0, 1.0)
    return throat * envelope

# -----
# Bernoulli gain
# -----

def _bernoulli_gain(self, r0: float) -> float:
    """ $G = 1 - (r_{in}/r_0)^2$  - pressure drop amplification at throat."""
    return max(0.0, 1.0 - (self.cfg.r_in / max(r0, self.cfg.r_in + 1e-8))**2)

# -----
# Throat evolution
# -----

def _evolve_throat(self, E_field: float, dt: float) -> Tuple[float, float]:
    """
     $dr_0/dt = -\lambda_v \cdot (E - E^*)/E^* \cdot r_0$ 

     $E > E^*$ : throat contracts (increases suction, pulls more domain in)
     $E < E^*$ : throat expands (suction releases, system near closure)
    """
    c = self.cfg
    E_star = silver_E_target(c.kT)
    dr0_dt = -c.lambda_v * (E_field - E_star) / (E_star + 1e-15) * self._stat
    r0_new = np.clip(self._state.r0 + dr0_dt * dt, c.r0_min, c.r0_max)
    return float(r0_new), float(dr0_dt)

# -----
# Main step

```

```

# -----

def step(
    self,
    psi: np.ndarray,          # GENERIC field (N×N complex)
    psi_inject: np.ndarray,   # injection field – any domain (N×N complex)
    z_dc: np.ndarray,        # DC Fourier mode (projected state)
    dt: float,
    center: Optional[Tuple[float, float]] = None,
) -> Tuple[np.ndarray, VenturiState]:
    """
    Apply one Venturi step.

    Parameters
    -----
    psi:          Current GENERIC field
    psi_inject:   Domain signal to inject (semantic field, MSR response field,
    z_dc:         DC projected state (drives E and throat evolution)
    dt:           Timestep
    center:       Optional (cx, cy) to shift throat center. If None, follows
                  field center of mass.

    Returns
    -----
    psi_out:      Field with Venturi injection applied
    state:        Updated VenturiState
    """
    c = self.cfg

    # Current DC energy
    E_field = float(np.real(z_dc @ z_dc) / 2.0)

    # Evolve throat radius
    r0_new, dr0_dt = self._evolve_throat(E_field, dt)
    self._state.r0 = r0_new
    self._state.G = self._bernoulli_gain(r0_new)
    self._state.E_field = E_field
    self._state.t += dt
    self._state.dr0_dt = dr0_dt

    # Optionally re-center the Venturi on field center of mass
    R = self._get_R(psi, center)

    # Venturi coupling field (shape: N×N, real)
    V = self._venturi_field_from_R(R, r0_new)

    # Coupling differential:  $d\psi/dt|_V = G \cdot V \cdot \psi_{\text{inject}}$ 

```

```

# Applied as Euler step: psi_out = psi + dt * G * coupling * V * psi_inje
dpsi = dt * self._state.G * c.coupling * V * psi_inject

psi_out = psi + dpsi

self._state.record()
return psi_out, self._state

# -----
# Centered radial grid (follows field center of mass)
# -----

def _get_R(self, psi: np.ndarray, center=None) -> np.ndarray:
    """
    If center is None: compute from field intensity center of mass.
    Otherwise: use supplied (cx, cy).
    The Venturi is always 360-degree around this center.
    """
    if center is not None:
        cx, cy = center
    else:
        # Center of mass of field intensity
        intensity = np.abs(psi)**2
        total = intensity.sum() + 1e-15
        x = np.linspace(0, 1, self.N, endpoint=False)
        cx = float((intensity * x[None, :]).sum() / total)
        cy = float((intensity * x[:, None]).sum() / total)

    x = np.linspace(0, 1, self.N, endpoint=False)
    X, Y = np.meshgrid(x, x)
    dx = np.abs(X - cx); dx = np.minimum(dx, 1 - dx)
    dy = np.abs(Y - cy); dy = np.minimum(dy, 1 - dy)
    return np.sqrt(dx**2 + dy**2)

def _venturi_field_from_R(self, R: np.ndarray, r0: float) -> np.ndarray:
    c = self.cfg
    r_eff = np.maximum(R, c.r_in)
    throat = (r0 / r_eff) ** c.beta
    envelope = np.clip((c.r_out - R) / (c.r_out - r0 + 1e-8), 0.0, 1.0)
    return throat * envelope

# -----
# Properties
# -----

@property
def state(self) -> VenturiState:

```

```
        return self._state

@property
def r0(self) -> float:
    return self._state.r0

@property
def gain(self) -> float:
    return self._state.G

def reset(self):
    cfg = self.cfg
    self._state = VenturiState(
        r0=cfg.r0_init, G=self._bernoulli_gain(cfg.r0_init),
        E_field=0.0, t=0.0, dr0_dt=0.0,
    )
```